

Universidad Veracruzana



Facultad de Estadística e Informática

**Licenciatura en Ingeniería de Ciberseguridad
e Infraestructura de Cómputo**

Fascículo de Estudio – Parte Practica

Experiencia Educativa: Programación en Red

Académico: Jose Pablo Rueda Holguín

Alumno: Javier Patiño González **Equipo:** 1

Fecha: 22 de septiembre de 2026

- Sección A0 – Repaso de Python (subraya la letra)

A0.1. En Python, PUERTO = 5050 crea:

- | | |
|---------------------------------------|-----------------|
| a) una función | c) una librería |
| <u>b) una variable de tipo entero</u> | d) un socket |

A0.2. b"holá" es un valor de tipo:

- | | |
|--------|-----------------|
| a) str | <u>c) bytes</u> |
| b) int | d) bool |

A0.3. El ciclo while True: termina cuando:

- | | |
|-------------------------------|-------------------------|
| a) llega a 10 vueltas | c) se acaba la memoria |
| <u>b) se ejecuta un break</u> | d) nunca puede terminar |

A0.4. conexion, direccion = servidor.accept() funciona porque accept() devuelve:

- | | |
|-------------------|--------------------------------------|
| a) una lista | <u>c) una tupla de dos elementos</u> |
| b) un diccionario | d) un bool |

A0.5. Un entorno virtual (venv) sirve para:

- | | |
|---|--------------------------|
| a) hacer Python más rápido | c) conectarse a internet |
| <u>b) aislar las librerías y versiones de cada proyecto</u> | d) cifrar el código |

- Sección A – Opción Múltiple (encierra la letra)

A1. Un sistema distribuido se caracteriza porque sus componentes:

- | | |
|---|-------------------------------|
| a) comparten la misma memoria RAM | c) corren en la misma máquina |
| <u>b) están en computadoras conectadas por red y se comunican solo por paso de mensajes</u> | d) no necesitan protocolos |

A2. En el modelo cliente-servidor:

- | | |
|--|----------------------------------|
| a) el servidor inicia la comunicación | c) ambos inician al mismo tiempo |
| <u>b) el cliente inicia y el servidor espera en una dirección conocida</u> | d) no hay direcciones conocidas |

A3. ¿Cuántos bytes puede representar un valor de 8 bits (un byte)?

- | | |
|--------|---------------|
| a) 8 | <u>c) 256</u> |
| b) 127 | d) 1024 |

A4. El número ASCII 10 corresponde a:

- | | |
|---------------|----------------------------------|
| a) el espacio | <u>c) el salto de línea (\n)</u> |
| b) la letra A | d) el dígito 0 |

A5. El puerto bien conocido de HTTPS es:

- | | |
|-------|---------------|
| a) 80 | <u>c) 443</u> |
| b) 22 | d) 53 |

A6. La llamada accept() devuelve:

- | | |
|--|-------------------------------------|
| a) solo la dirección del cliente | c) el número de clientes conectados |
| <u>b) un socket nuevo para hablar con ese cliente y su dirección</u> | d) el backlog disponible |

A7. Los tres mensajes del handshake TCP, en orden:

- | | |
|-----------------------------|--------------------|
| a) ACK, SYN, SYN-ACK | c) SYN, ACK, FIN |
| <u>b) SYN, SYN-ACK, ACK</u> | d) HELLO, SYN, ACK |

A8. recv(1024) significa:

- | | |
|----------------------------------|--|
| a) recibe exactamente 1024 bytes | <u>c) recibe HASTA 1024 bytes de lo que haya llegado</u> |
| b) recibe exactamente un mensaje | d) recibe 1024 mensajes |

A9. En UDP, la dirección del remitente se obtiene con:

- | | |
|--------------|----------------------|
| a) accept() | <u>c) recvfrom()</u> |
| b) connect() | d) bind() |

A10. Para enviar un ARCHIVO de 3 MB de forma confiable, el transporte adecuado es:

- | | |
|---|--|
| a) UDP, porque es más rápido | c) da igual |
| <u>b) TCP, porque garantiza entrega y orden</u> | d) ninguno, los archivos no viajan por red |

A11. Para video EN TIEMPO REAL, el transporte adecuado es:

- | | |
|--|-------------|
| a) TCP, porque retransmite los cuadros perdidos | c) da igual |
| <u>b) UDP, porque un cuadro perdido retransmitido llegaría tarde</u> | d) Telnet |

A12. El propósito principal de RPC es:

- | | |
|----------------------------|---|
| a) cifrar mensajes | <u>c) transparencia de acceso: llamar lo remoto como si fuera local</u> |
| b) hacer la red más rápida | d) eliminar el sistema operativo |

- **Seccion B – Verdadero o Falso (Encierra y justifica en la linea)**

B1. En un sistema distribuido, una parte puede fallar mientras las demás siguen funcionando; tu programa debe contemplarlo. V / F

Verdadero, al tratarse de un sistema distribuido, hay varias instancias de este aplicativo corriendo en paralelo, además de que, si el servicio dejase de funcionar, los otros pueden seguir operando sin él, dentro de sus capacidades.

B2. La memoria compartida es el modelo natural para comunicar máquinas distintas por red. V / F

Falso, el uso de la memoria es independiente a la comunicación en red.

B3. 1 KB equivale a 1000 bytes exactamente. V / F

Falso, 1KB equivale a 1024 bytes.

B4. El carácter ñ ocupa exactamente 1 byte en UTF-8. V / F

Falso, ñ requiere de al menos 2 bytes.

B5. El socket que devuelve accept() es el mismo que hizo listen(). V / F

Falso, el socket que se utiliza en listen() solo sirve para entablar la comunicación, el que devuelve accept() es por donde atraviesa la información.

B6. TCP garantiza que cada send() corresponde a exactamente un recv(). V / F

Verdadero, cada función de send() debe estar emparejado a una función de recv().

B7. Si un datagrama UDP se pierde, el protocolo lo retransmite automáticamente. V / F

Falso, el protocolo UDP no prioriza que la información llegue completa, solo que llegue rápido.

B8. UDP es ideal para aplicaciones que no pueden permitir la pérdida de datos. V / F

Falso, al contrario, UDP es idóneo en entornos donde se pueda permitir la pérdida de datos por la necesidad de ser transmitidos rápido.

Seccion C – Relacionar Columnas (escribe la letra)

Llamada	Respuesta	Descripción
1. socket()	c	a. Asocia el socket a una dirección y puerto locales
2. bind()	a	b. Inicia la conexión hacia el servidor (dispara el handshake)
3. listen()	e	c. Crea el punto final de comunicación
4. accept()	h	d. Envía un datagrama indicando el destinatario cada vez
5. connect()	b	e. Pone el socket en modo escucha
6. recvfrom()	g	f. Cierra el punto final de comunicación
7. sendto()	d	g. Recibe un datagrama y la dirección de quien lo envió
8. close()	f	h. Acepta una conexión y devuelve un socket nuevo + dirección

Seccion D – Respuesta corta (usa las lineas)

D1. Define con tus palabras “sistema distribuido” y menciona DOS problemas que no existen en una sola máquina.

Conjunto de dispositivos conectados en una red que en conjunto ofrecen una serie de servicios a usuarios como si se tratase de un solo servidor. Algunas ventajas de los equipos individuales es que existe menos retraso en el envío de mensajes y que es mas controlable por ser un solo dispositivo.

D2. Explica la regla de oro “ENVIAR = ENCODE · RECIBIR = DECODE” y qué error aparece si se olvida cada mitad.

Simplemente se trata sobre el envío de bytes en la red, cuando se trata de enviar un mensaje, primero debe ser Codificado a un formato de bytes, para que cuando llegue, deba ser Decodificado, en un formato legible. Olvidarlo únicamente haría que la información llegue ya sea corrompida por la falta de codificación, o se vuelva ilegible por la falta de decodificación.

D3. ¿Por qué el servidor necesita bind() y el cliente no? ¿Qué hace el sistema operativo por el cliente?

Porque el servidor necesita tener una dirección estable y consistente durante la comunicación, no puede darse el lujo de cambiar constantemente de ip y de puerto, mientras que el usuario puede llegar de diferentes direcciones, con un puerto de regreso generado dinámicamente por el propio sistema, por lo que, durante la comunicación, el sistema toma estos recursos y los envía automáticamente.

D4. Explica el problema del framing y describe DOS soluciones.

El framing es el problema que se genera durante una comunicación por sockets, pues la función recv() debe ser configurado con un buffer específico, buffer que debe ser llenado, esto hace que cuando se envíe un mensaje que no llene el buffer, el siguiente mensaje puede llegar cortado. Algunas soluciones vendrían siendo llenar el buffer después de que se use con caracteres vacíos, o enviar el mensaje con una longitud específica para que se ajuste al buffer.

D5. ¿Qué son stub y skeleton en RPC, en qué máquina va cada uno, y qué es marshalling?

Stub hace referencia al cliente y Skeleton hace referencia al servidor en una comunicación. Marshalling es el acto de convertir los datos en un formato transmitible por la red.

Seccion E – Lectura de Código

E1. ¿Qué imprime exactamente este programa cuando un cliente se conecta y envía "hola"?

```
import socket
servidor = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
servidor.bind(("localhost", 5050))
servidor.listen(1)
conexion, direccion = servidor.accept()
datos = conexion.recv(1024)
print("Me llego:", datos)
print("Tipo:", type(datos))
```

Imprime lo siguiente:

```
Me llego: b'hola'
Tipo: <class 'bytes'>
```

Esto se debe a que jamás se decodifico la información con decode("UTF-8").

E2. Este código UDP tiene UN error conceptual. Encuéntralo y corrígelo:

```
import socket
servidor = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
servidor.bind(("0.0.0.0", 5050))
servidor.listen(1)      # línea con problema
datos, direccion = servidor.recvfrom(1024)
```

Al tratarse de un socket UDP, no requiere escuchar, por lo que no le sirve la función de listen(), si se elimina, funciona perfectamente.

```
import socket
servidor = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
servidor.bind(("0.0.0.0", 5050))
datos, direccion = servidor.recvfrom(1024)
```

E3. ¿Cuál es el error de este cliente TCP?

```
cliente = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
cliente.connect(("localhost", 5050))
cliente.send("hola servidor")
```

Se olvido de codificar el mensaje, requiere un encode("UTF-8") previo o dentro del send().

```
cliente = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
cliente.connect(("localhost", 5050))
cliente.send("hola servidor".encode("UTF-8"))
```

Seccion F – Ejercicios Practicos (en tu computadora; entrega capturas)

Cada ejercicio usa los pasos de ejecución en dos terminales del código correspondiente (Tema 20). Verifica la salida esperada antes de contestar.

F1. Ciclos de vida (Códigos 1 y 2). Ejecuta el eco TCP. En una copia del código, muestra:

(a) la línea donde el servidor se bloquea esperando cliente.

```
# 4) accept(): BLOCKING (bloqueante) -- el programa SE DETIENE aqui
# hasta que un cliente se conecte. Devuelve DOS cosas:
# un socket NUEVO para hablar con ese cliente, y su direccion.
conexion, direccion = servidor.accept()
```

(b) la línea donde se bloquea esperando datos.

```
datos = conexion.recv(1024) # recv tambien es bloqueante
```

(c) la línea del cliente que dispara el handshake.

```
# connect(): inicia la conexion (dispara el three-way handshake).
# Si nadie escucha en esa IP:puerto, lanza ConnectionRefusedError.
cliente.connect((IP_SERVIDOR, PUERTO))
```

F2. Ciclo while True + funciones (Código 1). Modifica el servidor de eco: agrega un contador y una función armar_eco(mensaje, numero) que devuelva ECO #N> mensaje. Prueba con tres mensajes.

```
def armar_eco(mensaje, numero):
    return f"ECO #{numero}> " + mensaje
```

```
cont = 1
while True:
    datos = conexion.recv(1024) # recv tambien es bloqueante
    if not datos: # datos vacios (b'') = el cliente cerro
        break
    mensaje = datos.decode("utf-8") # RECIBIR = DECODE
    print("Me llevo:", mensaje)
    eco = armar_eco(mensaje, cont)
    conexion.send(eco.encode("utf-8")) # ENVIAR = ENCODE
    cont += 1
```

```
Conectado al servidor eco. Escriba mensajes (vacío para salir).
Usted dice: hola
El servidor responde: ECO #1> hola
Usted dice: turno
El servidor responde: ECO #2> turno
Usted dice: vespertino
El servidor responde: ECO #3> vespertino
```

F3. UDP y best-effort (Código 4). Ejecuta el experimento de pérdida (apagar el servidor a la mitad). Documenta con captura qué observó el cliente y explica en 3 líneas por qué no hubo ningún error de conexión.

Cuando el servidor se apaga durante la conexión, el cliente no lo presencia hasta que se trata de enviar un mensaje, donde se puede ver un error donde trata de recibir información de una conexión ya no existente.

```
C:\Users\JavoPC\Desktop>py "UDP eco cliente.py"
Eco UDP. Escriba mensajes (vacío para salir).
Usted dice: hola
El servidor responde: ECO> hola
Usted dice: eo
Traceback (most recent call last):
  File "C:\Users\JavoPC\Desktop\UDP eco cliente.py", line 18, in <module>
    datos, _ = cliente.recvfrom(1024) # RECIBIR = DECODE
                ~~~~~^~~~~~
ConnectionResetError: [WinError 10054] Se ha forzado la interrupción de un
a conexión existente por el host remoto
```

F4. Framing (Código 5). Ejecuta el servidor y cliente de framing. Luego modifica el cliente para enviar TRES mensajes en un solo send() y verifica que el servidor los separe. ¿Qué línea del servidor es la que “sabe” dónde termina cada mensaje?

```
cliente.send("primer mensaje\nsegundo mensaje\ntercer mensaje\n".encode("utf-8"))
```

Cuando llega información, se guarda en una variable llamada “buffer”, la cual dentro de un while(), corta el mensaje cuando encuentra un “\n” y muestra individualmente cada mensaje.

```
# mientras haya un delimitador en el buffer, hay un mensaje COMPLETO
while "\n" in buffer:
    mensaje, buffer = buffer.split("\n", 1) # corta el primer mensaje
    print("servidor framing> mensaje completo recibido:", mensaje)
```

F5. Bloqueo (Código 3). Cambia ESPERA a 7 segundos, ejecuta y anota el tiempo que midió el servidor. Después agrega `conexion.settimeout(5)` justo después de `accept()` y vuelve a correr: ¿qué excepción ocurre y en qué lado?

Ocorre un Time Out del lado del servidor, se debe a que la variable que controla cuanto tiempo puede pasar sin conexión fue de 5 segundos, y si el mensaje tarda 7 en ser enviado, el servidor se cansa y cierra antes.

```
servidor> esperando conexion...
servidor> cliente conectado: ('127.0.0.1', 52740)
servidor> voy a llamar recv() ... (aquí me BLOQUEO hasta que llegue algo)
Traceback (most recent call last):
  File "C:\Users\JavoPC\Desktop\bloqueo.py", line 16, in <module>
    datos = conexion.recv(1024) # <-- AQUÍ OCURRE EL BLOQUEO
TimeoutError: timed out
```

F6. Código limpio. Reescribe este código cumpliendo TODAS las normas de código limpio (constantes, nombres, comentarios que explican el porqué):

```
s=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
s.bind('',9000)
s.listen(1)
c,a=s.accept()
d=c.recv(1024)
print(d)
```

Versión reescrita:

```
1  # Espacio de Variables y Constantes
2  # - Es recomendable usar constantes para valores que requieran
3  #   ser permutados antes de la ejecución del programa
4  # - Las constantes deben estar en mayúscula
5  PUERTO = 9000
6
7  # Inicio del socket de servidor
8  # - Para mayor claridad, es importante ponerle nombre reconocible
9  #   a todas las variables que se utilicen
10 # - Se debe separar con espacio las variables/funciones
11 servidor = socket.socket(socket.AF_INET,socket.SOCK_STREAM)
12 servidor.bind(('localhost', PUERTO))
13 servidor.listen(1)
14
15 # Conexión del socket
16 # - Se requiere separar y nombrar bien la tupla de variables
17 conexion, direccion = servidor.accept()
18 datos = conexion.recv(1024)
19
20 # Uso de información
21 # - Se debe decodificar antes de ser usada
22 print(datos.decode("UTF-8"))
```